

2025

Temas y Screens



Joaquín Salvador Martín

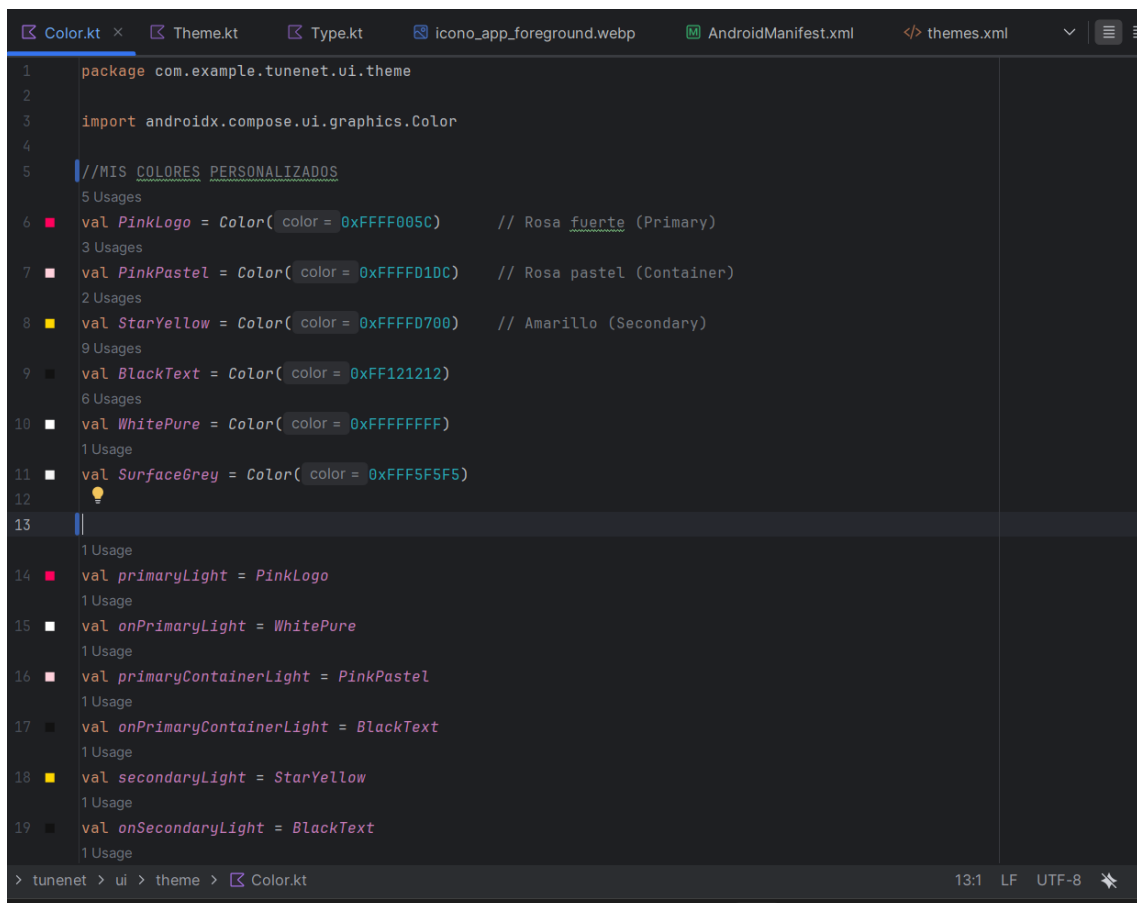
IES LA MARISMA

11-12-2025

Crea un tema completamente personalizado para esta app según las siguientes restricciones:

- Los colores deben estar basados en los del logo de la app.
- Se pueden realizar adaptaciones de colores de los principales roles a definir.
- Se deben crear al menos dos colores personalizados para usarlos en algún elemento.
- Especifica las fuentes usadas para la app y porqué las has usado, es decir, que tipo de fuente has buscado en base a qué criterios.

Los colores en los que me he basado del logo han sido estos ;



```
1 package com.example.tunenet.ui.theme
2
3 import androidx.compose.ui.graphics.Color
4
5 //MIS COLORES PERSONALIZADOS
6 val PinkLogo = Color( color = 0xFFFFF005C) // Rosa fuente (Primary)
7 val PinkPastel = Color( color = 0xFFFFD1DC) // Rosa pastel (Container)
8 val StarYellow = Color( color = 0xFFFFD700) // Amarillo (Secondary)
9 val BlackText = Color( color = 0xFF121212)
10 val WhitePure = Color( color = 0xFFFFFFFF)
11 val SurfaceGrey = Color( color = 0xFFFF5F5F)
12
13
14 val primaryLight = PinkLogo
15 val onPrimaryLight = WhitePure
16 val primaryContainerLight = PinkPastel
17 val onPrimaryContainerLight = BlackText
18 val secondaryLight = StarYellow
19 val onSecondaryLight = BlackText
```

De momento uso pocos porque la aplicación no tiene ni un 10 % realizado aun...

Este es la parte del Theme;

```
1 package com.example.tunenet.ui.theme
2
3 > import ...
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18 private val lightScheme = lightColorScheme(
19     primary = primaryLight,
20     onPrimary = onPrimaryLight,
21     primaryContainer = primaryContainerLight,
22     onPrimaryContainer = onPrimaryContainerLight,
23     secondary = secondaryLight,
24     onSecondary = onSecondaryLight,
25     secondaryContainer = secondaryContainerLight,
26     onSecondaryContainer = onSecondaryContainerLight,
27     tertiary = tertiaryLight,
28     onTertiary = onTertiaryLight,
29     tertiaryContainer = tertiaryContainerLight,
30     onTertiaryContainer = onTertiaryContainerLight,
31     error = errorLight,
32     onError = onErrorLight,
33     errorContainer = errorContainerLight,
34     onErrorContainer = onErrorContainerLight,
35     background = backgroundLight,
36     onBackground = onBackgroundLight,
37     surface = surfaceLight,
38     onSurface = onSurfaceLight,
39     surfaceVariant = surfaceVariantLight,
40     onSurfaceVariant = onSurfaceVariantLight,
41     outline = outlineLight
42 )
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
```

```
44 private val darkScheme = darkColorScheme(
45     primary = primaryDark,
46     onPrimary = onPrimaryDark,
47     primaryContainer = primaryContainerDark,
48     onPrimaryContainer = onPrimaryContainerDark,
49     secondary = secondaryDark,
50     onSecondary = onSecondaryDark,
51     secondaryContainer = secondaryContainerDark,
52     onSecondaryContainer = onSecondaryContainerDark,
53     tertiary = tertiaryDark,
54     onTertiary = onTertiaryDark,
55     tertiaryContainer = tertiaryContainerDark,
56     onTertiaryContainer = onTertiaryContainerDark,
57     error = errorDark,
58     onError = onErrorDark,
59     errorContainer = errorContainerDark,
60     onErrorContainer = onErrorContainerDark,
61     background = backgroundDark,
62     onBackground = onBackgroundDark,
63     surface = surfaceDark,
64     onSurface = onSurfaceDark,
65     surfaceVariant = surfaceVariantDark,
66     onSurfaceVariant = onSurfaceVariantDark,
67     outline = outlineDark
68 )
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
```

```

@Composable
fun AppTheme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    dynamicColor: Boolean = false,
    content: @Composable () -> Unit
) {
    val colorScheme = when {
        dynamicColor && Build.VERSION.SDK_INT >= Build.VERSION_CODES.S -> {
            val context = LocalContext.current
            if (darkTheme) dynamicDarkColorScheme(context) else dynamicLightColorScheme(context)
        }
        darkTheme -> darkScheme
        else -> lightScheme
    }

    val view = LocalView.current
    if (!view.isInEditMode) {
        SideEffect {
            val window = (view.context as Activity).window
            // Usamos tu color primario (Rosa) para la barra de estado
            window.statusBarColor = colorScheme.primaryContainer.toArgb()
            WindowCompat.getInsetsController(window, view).isAppearanceLightStatusBars = !darkTheme
        }
    }

    MaterialTheme(
        colorScheme = colorScheme,
        typography = AppTypography,
        content = content
    )
}

```

Desactive el dynamicColor porque me daba errores a la hora de obtener mis colores principales del logo , y pense que haciendo false ese boolean se solucionaria y asi mismo se soluciono.

Type;

```
package com.example.tunenet.ui.theme

> import ...

2 Usages
val provider = GoogleFont.Provider(
    providerAuthority = "com.google.android.gms.fonts",
    providerPackage = "com.google.android.gms",
    certificates = R.array.com_google_android_gms_fonts_certs
) //Este es para que basicamente pueda usar imagenes url , tuve que importar un certificado en
// build.gradle.kts(Module:app) = Para que no me dieran errores , y asi no tener que depender de descargar imagenes

6 Usages
val bodyFontFamily = FontFamily(
    ..fonts = Font(
        googleFont = GoogleFont( name = "Nunito Sans"),
        fontProvider = provider,
    )
)

9 Usages
val displayFontFamily = FontFamily(
    ..fonts = Font(
        googleFont = GoogleFont( name = "Nunito"),
        fontProvider = provider,
    )
)

15 Usages
val baseline = Typography()

1 Usage
val AppTypography = Typography(
    displayLarge = baseline.displayLarge.copy(fontFamily = displayFontFamily),
    displayMedium = baseline.displayMedium.copy(fontFamily = displayFontFamily),
    displaySmall = baseline.displaySmall.copy(fontFamily = displayFontFamily),
    headlineLarge = baseline.headlineLarge.copy(fontFamily = displayFontFamily),
    headlineMedium = baseline.headlineMedium.copy(fontFamily = displayFontFamily),
    headlineSmall = baseline.headlineSmall.copy(fontFamily = displayFontFamily),
    titleLarge = baseline.titleLarge.copy(fontFamily = displayFontFamily),
    titleMedium = baseline.titleMedium.copy(fontFamily = displayFontFamily),
    titleSmall = baseline.titleSmall.copy(fontFamily = displayFontFamily),
    bodyLarge = baseline.bodyLarge.copy(fontFamily = bodyFontFamily),
    bodyMedium = baseline.bodyMedium.copy(fontFamily = bodyFontFamily),
    bodySmall = baseline.bodySmall.copy(fontFamily = bodyFontFamily),
    labelLarge = baseline.labelLarge.copy(fontFamily = bodyFontFamily),
    labelMedium = baseline.labelMedium.copy(fontFamily = bodyFontFamily),
    labelSmall = baseline.labelSmall.copy(fontFamily = bodyFontFamily),
) //Esta es la tipografia de mi app
```

Con este logro poner a disposición mi galería de tipografías , y mi certificado de Google para que no me de errores a la hora de usar imágenes con URLs...

Muestra y explica la creación de un par de componentes con estilos personalizados, como la tarjeta de elementos y otro componente de tu elección.

Para dotar mi app de colores consistentes , he creado componentes reutilizables usando “**Jetpack Compose**”, usando de “**AppTheme**” temas personalizados para así asegurar su tipografía y los colores de la app.

ELEMENTO DE LA LISTA (“TuneElementCard”)

Parte del móvil;

```
// 4. COMPONENTES Y TARJETAS
//Tarjeta Horizontal para Móvil
2 Usages
@Composable
fun TuneElementCard(item: MusicItem, onClick: () -> Unit, onFavClick: (MusicItem) -> Unit) {
    Card(modifier = Modifier.fillMaxWidth().padding(vertical = 8.dp).clickable { onClick() },
        elevation = CardDefaults.cardElevation(defaultElevation = 2.dp),
        colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surfaceVariant)) {
        Row(modifier = Modifier.padding( all = 16.dp), verticalAlignment = Alignment.CenterVertically) {
            AsyncImage(model = item.coverUrl, contentDescription = null, contentScale = ContentScale.Crop,
                alignment = Alignment.TopStart, modifier = Modifier.size( size = 64.dp).clip( shape = RoundedCornerShape( size = 8.dp)).background(Color.LightGray))
            Spacer(modifier = Modifier.width( width = 16.dp))
            Column(modifier = Modifier.weight( weight = 1f)) {
                Text( text = item.title, style = MaterialTheme.typography.titleMedium, fontWeight = FontWeight.Bold, maxLines = 1)
                Text( text = item.artist, style = MaterialTheme.typography.bodyMedium)
            }
            IconButton(onClick = { onFavClick(item) }) {
                Icon(imageVector = Icons.Filled.Favorite, contentDescription = null,
                    tint = if (item.isFavorite) MaterialTheme.colorScheme.primary else Color.Gray)
            }
        }
    }
}
```

Esta es para la parte visual del móvil una tarjeta horizontal en la parte de abajo para que se vea bonito al ojo humano

La parte para la Tablet es esta;

```
//Tarjeta Vertical para la Tablet
2 Usages
@Composable
fun TuneGridCard(item: MusicItem, onClick: () -> Unit, onFavClick: (MusicItem) -> Unit) {
    Card(modifier = Modifier.fillMaxWidth().clickable { onClick() },
        elevation = CardDefaults.cardElevation(defaultElevation = 4.dp),
        colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surfaceVariant)) {
        Column(horizontalAlignment = Alignment.CenterHorizontally) {
            AsyncImage(
                model = item.coverUrl,
                contentDescription = null,
                contentScale = ContentScale.Crop,
                modifier = Modifier.fillMaxWidth().height( height = 140.dp).background(Color.LightGray)
            )
            Column(modifier = Modifier.padding( all = 12.dp),
                horizontalAlignment = Alignment.CenterHorizontally
            ) {
                Text(text = item.title, style = MaterialTheme.typography.titleMedium, fontWeight = FontWeight.Bold, maxLines = 1, overflow = TextOverflow.Ellipsis)
                Text(text = item.artist, style = MaterialTheme.typography.bodySmall, maxLines = 1)
                Spacer(modifier = Modifier.height( height = 8.dp))

                IconButton(onClick = { onFavClick(item) }) {
                    Icon(imageVector = Icons.Filled.Favorite,
                        contentDescription = null,
                        tint = if (item.isFavorite) MaterialTheme.colorScheme.primary else Color.Gray)
                }
            }
        }
    }
}
```

La parte para la Tablet es una versión diferente como se pide en el ejercicio , en este caso la hacemos vertical...

Incluye lo mismo que la parte del móvil pero se aloja en otra zona de la pantalla.

Justificación de diseño y estilos:

- **Contenedor:** Se usa una **Card** con elevación suave (**2.dp**) y un color de fondo **surfaceVariant** (gris suave/rosado según el tema) para destacar sobre el fondo de la pantalla.
- **Layout:** Se utiliza un **Row (fila)** para alinear la carátula a la izquierda, la información en el centro y la acción a la derecha, siguiendo los patrones de diseño de apps de música como Spotify.
- **Estilo de Imagen:** La carátula tiene un **recorte (clip)** con **RoundedCornerShape(8.dp)** para suavizar las esquinas y hacerla más amigable.
- **Feedback Visual:** El icono del corazón cambia su propiedad **tint** dinámicamente: usa el color **Primary (Rosa)** cuando está activo y gris cuando no, **dando feedback inmediato al usuario**.

CABECERA DE DETALLE (“DetailHeader”);

Este componente es el elemento visual principal de la pantalla de detalles.

Con él lo que intento es impactar visualmente al usuario mostrando la portada del álbum en gran tamaño.

```
2 Usages
@Composable
fun DetailHeader(item: MusicItem) {
    AsyncImage(
        model = item.coverUrl, contentDescription = "Portada", contentScale = ContentScale.Crop, alignment = Alignment.TopStart,
        modifier = Modifier.fillMaxWidth().height(height = 300.dp).clip(shape = RoundedCornerShape(bottomStart = 32.dp, bottomEnd = 32.dp)).background(Color.LightGray)
    )
    Column(modifier = Modifier.padding(all = 24.dp)) {
        Text(text = item.title, style = MaterialTheme.typography.headlineLarge, fontWeight = FontWeight.Bold)
        Text(text = item.artist, style = MaterialTheme.typography.titleLarge, color = MaterialTheme.colorScheme.primary)
        Spacer(modifier = Modifier.height(height = 8.dp)); SuggestionChip(onClick = {}, label = { Text(text = item.genre) })
        Spacer(modifier = Modifier.height(height = 16.dp)); Text(text = item.description, style = MaterialTheme.typography.bodyLarge)
    }
}
```

Análisis de estilos aplicados:

- **Geometría Personalizada:** Se aplica un modificador “**clip(RoundedCornerShape(bottomStart = 32.dp, bottomEnd = 32.dp))**”.

Esto mantiene los bordes superiores rectos pero curva los inferiores, creando una transición estética hacia el texto.

- **Escala:** Se fija una altura de “**300.dp**” ocupando todo el ancho para dar protagonismo a la carátula.
- **Jerarquía:** Se usan los estilos de texto “**headlineLarge (título)**” y “**titleLarge (artista)**”, aplicando el color “**Primary (Rosa)**” al artista para vincularlo con la identidad de la app.

LA APP DEBE TENER LAS SIGUIENTES PANTALLAS (SCREENS)

EN FORMATO COMPACTO

ElemListScreen: Pantalla con la lista de elementos de los que trata la app. Cada elemento debe estar contenido en una tarjeta (Card). Esta tarjeta debe contener la información básica que estimes oportuna y botón para poder añadir a favoritos.

```
Usage
@Composable
fun ElemListScreen(musicItems: List<MusicItem>, isTablet: Boolean, onItemClick: (MusicItem) -> Unit, onFavClick: (MusicItem) -> Unit) {
    if (isTablet) {
        // MODO GRID PARA LA TABLE
        Column(modifier = Modifier.fillMaxSize().padding(all = 16.dp)) {
            Text(text = "Descubrir Música",
                style = MaterialTheme.typography.headlineMedium,
                color = MaterialTheme.colorScheme.primary, modifier = Modifier.padding(bottom = 16.dp))
            LazyVerticalGrid(
                columns = GridCells.Adaptive(minSize = 150.dp), // Celdas adaptables
                horizontalArrangement = Arrangement.spacedBy(space = 16.dp),
                verticalArrangement = Arrangement.spacedBy(space = 16.dp)
            ) {
                items(musicItems) { item -> TuneGridCard(item, onItemClick = { onItemClick(item) }, onFavClick) }
            }
        }
    } else {
        // MODO LISTA PARA EL MÓVIL
        LazyColumn(contentPadding = PaddingValues(all = 16.dp), modifier = Modifier.fillMaxSize()) {
            item { Text(text = "Descubrir Música",
                style = MaterialTheme.typography.headlineMedium,
                color = MaterialTheme.colorScheme.primary, modifier = Modifier.padding(bottom = 16.dp)) }
            items(musicItems) { item -> TuneElementCard(item, onItemClick = { onItemClick(item) }, onFavClick) }
        }
    }
}
```

DetailItemScreen: Pantalla con todos los datos detallados del elemento que se ha pulsado en la lista de elementos de la pantalla anterior. Aquí también debe aparecer un botón para incluir a favoritos.

```
Usage
@Composable
fun DetailItemScreen(item: MusicItem, onFavClick: () -> Unit) {
    Column(modifier = Modifier.fillMaxSize().verticalScroll(state = rememberScrollState()),
        horizontalAlignment = Alignment.CenterHorizontally)
    {
        DetailHeader(item)
        Spacer(modifier = Modifier.height(height = 24.dp))
        Button(
            onClick = onFavClick,
            colors = ButtonDefaults.buttonColors(containerColor = if (item.isFavorite) MaterialTheme.colorScheme.surfaceVariant else MaterialTheme.colorScheme.primary)
        ) {
            Icon(imageVector = if(item.isFavorite) Icons.Filled.Favorite else Icons.Filled.FavoriteBorder, contentDescription = null,
                tint = if(item.isFavorite) MaterialTheme.colorScheme.primary else Color.White)
            Spacer(modifier = Modifier.width(width = 8.dp))
            Text(text = if(item.isFavorite) "Quitar de favoritos" else "Añadir a favoritos",
                color = if (item.isFavorite) MaterialTheme.colorScheme.onSurfaceVariant else Color.White)
        }
    }
}
```

FavListScreen: Pantalla con la lista de elementos que han sido marcados como favoritos. En esta pantalla debe haber un botón para eliminar de favoritos.

```

fun FavListScreen(musicItems: List<MusicItem>, isTablet: Boolean, onItemClick: (MusicItem) -> Unit, onRemoveFav: (MusicItem) -> Unit) {
    if (isTablet) {
        // MODO GRID (TABLET)
        Column(modifier = Modifier.fillMaxSize().padding(all = 16.dp)) {
            Text(text = "Mis Favoritos",
                style = MaterialTheme.typography.headlineMedium,
                color = MaterialTheme.colorScheme.primary,
                modifier = Modifier.padding(bottom = 16.dp))
            if (musicItems.isEmpty())
                Text(text = "No tienes favoritos aún.",
                    style = MaterialTheme.typography.bodyLarge)
            LazyVerticalGrid(
                columns = GridCells.Adaptive(minSize = 150.dp),
                horizontalArrangement = Arrangement.spacedBy(space = 16.dp),
                verticalArrangement = Arrangement.spacedBy(space = 16.dp)
            ) {
                items(musicItems) { item -> TuneGridCard(item, onClick = { onItemClick(item) }, onFavClick = onRemoveFav) }
            }
        }
    } else {
        // MODO LISTA (MÓVIL)
        LazyColumn(contentPadding = PaddingValues(all = 16.dp),
            modifier = Modifier.fillMaxSize())
        {
            item { Text(text = "Mis Favoritos",
                style = MaterialTheme.typography.headlineMedium,
                color = MaterialTheme.colorScheme.primary,
                modifier = Modifier.padding(bottom = 16.dp)) }
            if (musicItems.isEmpty()) item { Text(text = "No tienes favoritos aún.", style = MaterialTheme.typography.bodyLarge) }
            items(musicItems) { item -> TuneElementCard(item, onClick = { onItemClick(item) }, onFavClick = onRemoveFav) }
        }
    }
}

```

DetailFavScreen: Pantalla de detalles del elemento favorito que se ha seleccionado de la lista. Debe mostrar una lista de comentarios asociado a este elemento y un botón FAB para permitir incluir nuevos comentarios.

```

@Composable
DetailFavScreen(item: MusicItem) {
    Column(modifier = Modifier.fillMaxSize().verticalScroll(state = rememberScrollState()),
        horizontalAlignment = Alignment.CenterHorizontally)
    {
        DetailHeader(item)
        Spacer(modifier = Modifier.height(height = 24.dp))
        HorizontalDivider()
        Text(
            text = "Comentarios",
            style = MaterialTheme.typography.titleMedium,
            modifier = Modifier.padding(all = 16.dp),
            color = MaterialTheme.colorScheme.primary
        )
        item.comments.forEach { comment ->
            Card(modifier = Modifier.fillMaxWidth().padding(horizontal = 16.dp, vertical = 4.dp),
                colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surfaceVariant))
            {
                Row(modifier = Modifier.padding(all = 16.dp),
                    verticalAlignment = Alignment.CenterVertically)
                {
                    Icon(imageVector = Icons.Filled.Person,
                        contentDescription = null,
                        modifier = Modifier.size(size = 24.dp))
                    Spacer(modifier = Modifier.width(width = 8.dp)); Text(text = comment)
                }
            }
        }
        Spacer(modifier = Modifier.height(height = 80.dp))
    }
}

```

ProfileScreen: Pantalla con información del usuario con un botón para hacer login/logout.

```
1 Usage
@Composable
fun ProfileScreen() {
    Column(modifier = Modifier.fillMaxSize().padding( all = 16.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center)
    {
        Icon(
            imageVector = Icons.Filled.Person,
            contentDescription = null,
            modifier = Modifier.size( size = 100.dp),
            tint = MaterialTheme.colorScheme.primary
        )
        Spacer(modifier = Modifier.height( height = 16.dp))

        Text( text = "Usuario: Joaquín", style = MaterialTheme.typography.headlineSmall)
        Text( text = "joaquinity@gmail.com", style = MaterialTheme.typography.bodyMedium)

        Spacer(modifier = Modifier.height( height = 32.dp))

        Button(
            onClick = { },
            colors = ButtonDefaults.buttonColors(containerColor = MaterialTheme.colorScheme.error)
        ) {
            Text( text = "Cerrar Sesión")
        }
    }
}
```

AboutScreen: Pantalla con información sobre la app.

```
@Composable
fun AboutScreen() {
    val context = LocalContext.current
    Column(modifier = Modifier.fillMaxSize().padding( all = 16.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center)
    {
        Text( text = "TuneNet", fontSize = 38.sp, color = MaterialTheme.colorScheme.primary, fontWeight = FontWeight.Bold)
        Text( text = "Versión: 2.2", style = MaterialTheme.typography.bodyLarge)
        Spacer(modifier = Modifier.height( height = 32.dp))
        Button(onClick =
            {
                val emailIntent = Intent( action = Intent.ACTION_SENDTO).apply {
                    data = Uri.parse( uriString = "mailto:");
                    putExtra( name = Intent.EXTRA_EMAIL,
                        value = arrayOf("joaquinity@gmail.com"))
                }

                try
                { context.startActivity(Intent.createChooser( target = emailIntent, title = "Enviar correo...")) }
                catch (e: Exception) {}

            })
        {
            Icon( imageVector = Icons.Filled.Email,
                contentDescription = null);
            Spacer(modifier = Modifier.width( width = 8.dp));
            Text( text = "Soporte")
        }
    }
}
```

Aquí muestra la versión de la app y un botón que si le das te lleva a correos para que envíes un mensaje a soporte , el cual es mi correo.

Implementación de Variantes de Interfaz (Responsive Design)

Para cumplir con los estándares del ejercicio se pide ofrecer una experiencia completa para (Tablet o móviles en horizontal) y para eso he implementado un sistema de “**Diseño Adaptativo**”.

El objetivo es evitar que el contenido se vea simplemente "estirado" en pantallas anchas. Para ello, he usado las “**Windows Size Classes**” de Material Design 3.

Detección del Entorno

En la MainActivity, se calcula el tamaño de la ventana actual para determinar una variable booleana **isTablet**. Esta variable actúa como bandera para cambiar la lógica de renderizado en toda la aplicación.

```
setContent {  
    AppTheme {  
        val windowSize = calculateWindowSizeClass(activity = this)  
        val isTablet = windowSize.widthSizeClass != WindowWidthSizeClass.Compact
```

Variante en Pantallas de Lista (ElemListScreen)

Esta es la transformación más significativa.

- **En Móvil (Compact):** Se utiliza una **LazyColumn**. Los elementos se muestran uno debajo del otro para facilitar el **scroll** vertical con una mano.

```
} else {  
    // MODO LISTA PARA EL MÓVIL  
    LazyColumn(contentPadding = PaddingValues(all = 16.dp), modifier = Modifier.fillMaxSize()) {  
        item { Text(text = "Descubrir Música",  
            style = MaterialTheme.typography.headlineMedium,  
            color = MaterialTheme.colorScheme.primary, modifier = Modifier.padding(bottom = 16.dp)) }  
        items(musicItems) { item -> TuneElementCard(item, onClick = { onItemClick(item) }, onFavClick) }  
    }  
}
```

- **En Tablet (Medium/Expanded):** Se cambia la estructura a un **LazyVerticalGrid**. Esto permite mostrar múltiples tarjetas por fila, aprovechando el ancho de la pantalla y evitando líneas de texto excesivamente largas y difíciles de leer.

```
fun ElemListScreen(musicItems: List<MusicItem>, isTablet: Boolean, onItemClick: (MusicItem) -> Unit, onFavClick: (MusicItem) -> Unit) {
    if (isTablet) {
        // MODO GRID PARA LA TABLE
        Column(modifier = Modifier.fillMaxSize().padding(all = 16.dp)) {
            Text(text = "Descubrir Música",
                style = MaterialTheme.typography.headlineMedium,
                color = MaterialTheme.colorScheme.primary, modifier = Modifier.padding(bottom = 16.dp))
            LazyVerticalGrid(
                columns = GridCells.Adaptive(minSize = 150.dp), // Celdas adaptables
                horizontalArrangement = Arrangement.spacedBy(space = 16.dp),
                verticalArrangement = Arrangement.spacedBy(space = 16.dp)
            ) {
                items(musicItems) { item -> TuneGridCard(item, onClick = { onItemClick(item) }, onFavClick) }
            }
        }
    }
}
```

Variante en Pantallas de Detalle y Navegación

Para las pantallas de detalle (**DetailItemScreen** y **DetailFavScreen**), la variante principal reside en la **Navegación Ergonómica**.

En pantallas expandidas, una barra de navegación inferior (**NavigationBar**) es incómoda visualmente y ocupa espacio vertical valioso. Por ello, se ha implementado un cambio dinámico en la estructura del **Scaffold**:

- **Móvil:** La navegación se mantiene en la parte inferior.
- **Tablet/Expandido:** La navegación se desplaza a un **Navigation Rail** lateral (columna izquierda).

Esto es más ergonómico para sujetar una Tablet con dos manos y deja todo el espacio vertical libre para el contenido del detalle (la imagen de cabecera y la descripción).

```
Row(modifier = Modifier.fillMaxSize()) {
    if (isTablet) {
        NavigationRail(containerColor = MaterialTheme.colorScheme.surfaceVariant) {
            Spacer(modifier = Modifier.weight(weight = 1f))
            NavigationRailItem(
                icon = { Icon(imageVector = Icons.Filled.Home, contentDescription = null) },
                label = { Text(text = "Inicio") },
                selected = currentScreen == CurrentScreen.LIST || currentScreen == CurrentScreen.DETAIL,
                onClick = { currentScreen = CurrentScreen.LIST }
            )
            NavigationRailItem(
                icon = { Icon(imageVector = Icons.Filled.Favorite, contentDescription = null) },
                label = { Text(text = "Favoritos") },
                selected = currentScreen == CurrentScreen.FAV_LIST || currentScreen == CurrentScreen.FAV_DETAIL,
                onClick = { currentScreen = CurrentScreen.FAV_LIST }
            )
            NavigationRailItem(
                icon = { Icon(imageVector = Icons.Filled.Person, contentDescription = null) },
                label = { Text(text = "Perfil") }, selected = currentScreen == CurrentScreen.PROFILE,
                onClick = { currentScreen = CurrentScreen.PROFILE }
            )
            Spacer(modifier = Modifier.weight(weight = 1f))
        }
    }
}
```

Configuración de la Automatización de Visualización según Tamaño de Ventana

Para lograr que la aplicación se adapte automáticamente a diferentes factores de forma (Móvil vertical, Tablet, Móvil horizontal), implemente la librería ;

- **androidx.compose.material3:material3-window-size-class**

Con esta configuración lo que consigo es que permita detectar las dimensiones de la ventana en tiempo de ejecución y así clasificar el dispositivo en uno de los tres anchos estándar de Material Design:

- **Compact** (Compacto)
- **Medium** (Medio)
- **Expanded** (Expandido).

Implementación en el Ciclo de Vida (MainActivity)

Dentro del onCreate de la actividad principal, se invoca la función **calculateWindowSizeClass(this)**.

Esta función retorna el estado actual de la ventana y se recalcula automáticamente si el usuario gira el dispositivo o cambia el tamaño de la ventana (en modo multiventana).

```
class MainActivity : ComponentActivity() {
    @OptIn(markerClass = ExperimentalMaterial3WindowSizeClassApi::class, ExperimentalMaterial3Api::class)
    override fun onCreate(savedInstanceState: Bundle?) {
        val splashScreen = installSplashScreen()
        super.onCreate(savedInstanceState)

        var keepSplashOnScreen = true
        val startTime = System.currentTimeMillis()
        splashScreen.setKeepOnScreenCondition { System.currentTimeMillis() - startTime < 2000L }
        splashScreen.setOnExitAnimationListener { splashScreenView ->
            val slideUp = ObjectAnimator.ofFloat(target = splashScreenView.iconView, property = View.TRANSLATION_Y, values = 0f, -splashScreenView.view.height.toFloat())
            slideUp.interpolator = AnticipateInterpolator(); slideUp.duration = 500L
            val fadeOut = ObjectAnimator.ofFloat(target = splashScreenView.view, property = View.ALPHA, values = 1f, 0f)
            fadeOut.duration = 500L; slideUp.start(); fadeOut.start()
            fadeOut.doOnEnd { splashScreenView.remove() }
        }

        enableEdgeToEdge()

        setContent {
            AppTheme {
                val windowSize = calculateWindowSizeClass(activity = this)
                val isTablet = windowSize.widthSizeClass != WindowWidthSizeClass.Compact

                var currentScreen by remember { mutableStateOf(value = CurrentScreen.LIST) }
                var selectedItem by remember { mutableStateOf<MusicItem?>(value = null) }
                val showFab = currentScreen == CurrentScreen.FAV_DETAIL

                Row(modifier = Modifier.fillMaxSize()) {
                    if (isTablet) {
                        NavigationRail(containerColor = MaterialTheme.colorScheme.surfaceVariant) {
```

Lógica de Decisión (Compacto vs Expandido)

Para simplificar la lógica de la interfaz, he automatizado la decisión binaria mediante la variable **isTablet**.

- **Modo Compacto (WindowWidthSizeClass.Compact):**

Se asocia al uso tradicional de smartphone en vertical. La app renderiza:

- Navegación inferior (**NavigationBar**).
- Listas verticales (**LazyColumn**).
- Tarjetas horizontales (**TuneElementCard**).

- **Modo Medio/Expandido (Medium / Expanded):**

Se asocia a Tablets o teléfonos grandes en horizontal. La app la transforma automáticamente para renderizar:

- Navegación lateral izquierda (**NavigationRail**) para mejor ergonomía.
- Cuadrículas (**LazyVerticalGrid**) para aprovechar el ancho de pantalla.
- Tarjetas verticales (**TuneGridCard**).

Ejemplo de Automatización en el Layout

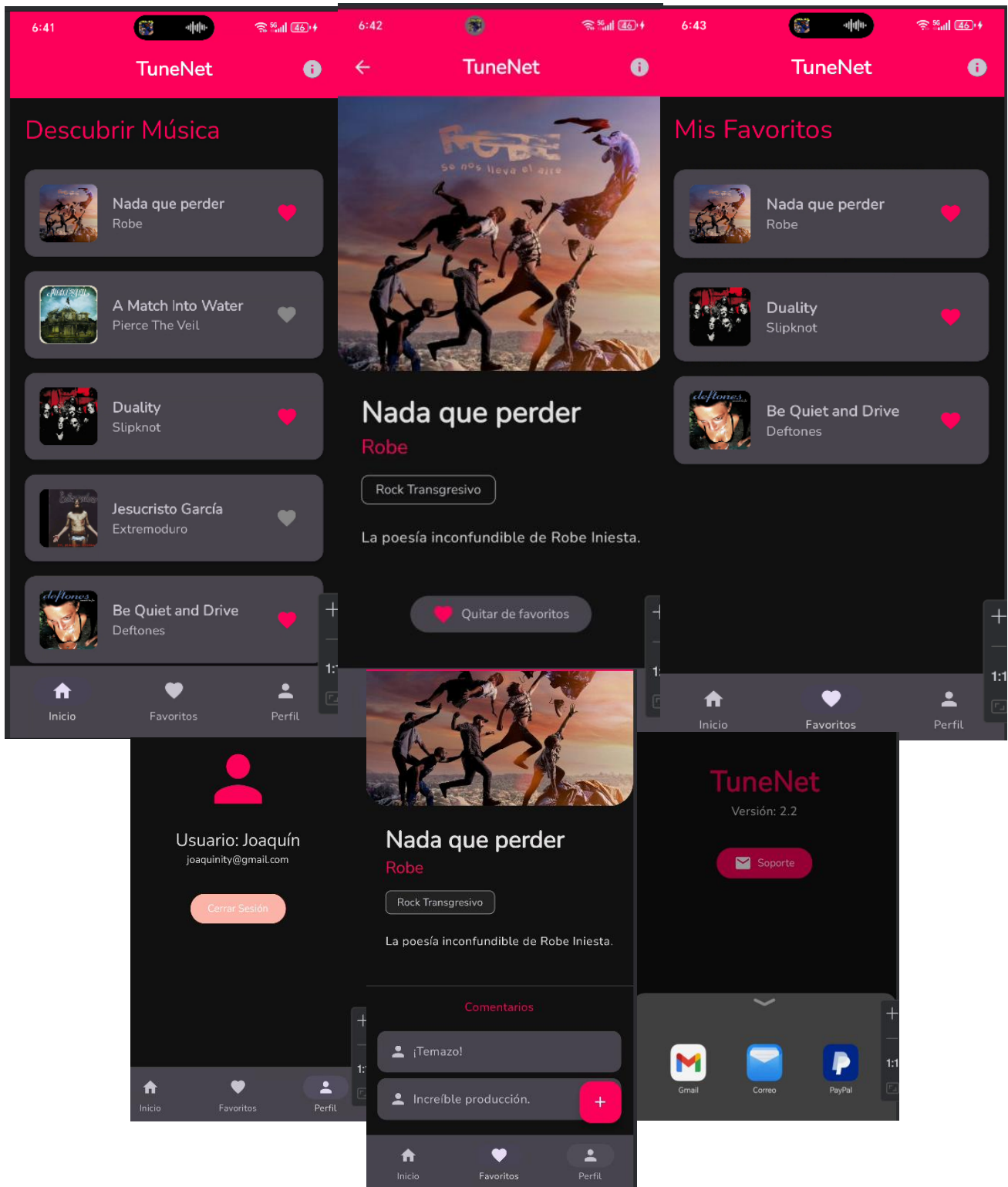
El contenedor principal de la aplicación reacciona a esta configuración para decidir qué barra de navegación mostrar sin intervención del usuario:

```
Row(modifier = Modifier.fillMaxSize()) {  
    if (isTablet) {  
        NavigationRail(containerColor = MaterialTheme.colorScheme.surfaceVariant) {  
            bottomBar = {  
                if (!isTablet) {  
                    NavigationBar(containerColor = MaterialTheme.colorScheme.surfaceVariant) {  
                        NavigationBarItem(  
                            icon = { Icon( imageVector = Icons.Filled.Home, contentDescription = null) },  
                            label = { Text( text = "Inicio") },  
                            selected = currentScreen == CurrentScreen.LIST || currentScreen == CurrentScreen.DETAIL,  
                            onClick = { currentScreen = CurrentScreen.LIST }  
                        )  
                        NavigationBarItem(  
                            icon = { Icon( imageVector = Icons.Filled.Favorite, contentDescription = null) },  
                            label = { Text( text = "Favoritos") },  
                            selected = currentScreen == CurrentScreen.FAV_LIST || currentScreen == CurrentScreen.FAV_DETAIL,  
                            onClick = { currentScreen = CurrentScreen.FAV_LIST }  
                        )  
                        NavigationBarItem(  
                            icon = { Icon( imageVector = Icons.Filled.Person, contentDescription = null) },  
                            label = { Text( text = "Perfil") },  
                            selected = currentScreen == CurrentScreen.PROFILE,  
                            onClick = { currentScreen = CurrentScreen.PROFILE }  
                        )  
                    }  
                }  
            }  
        }  
    },  
    },  
}
```

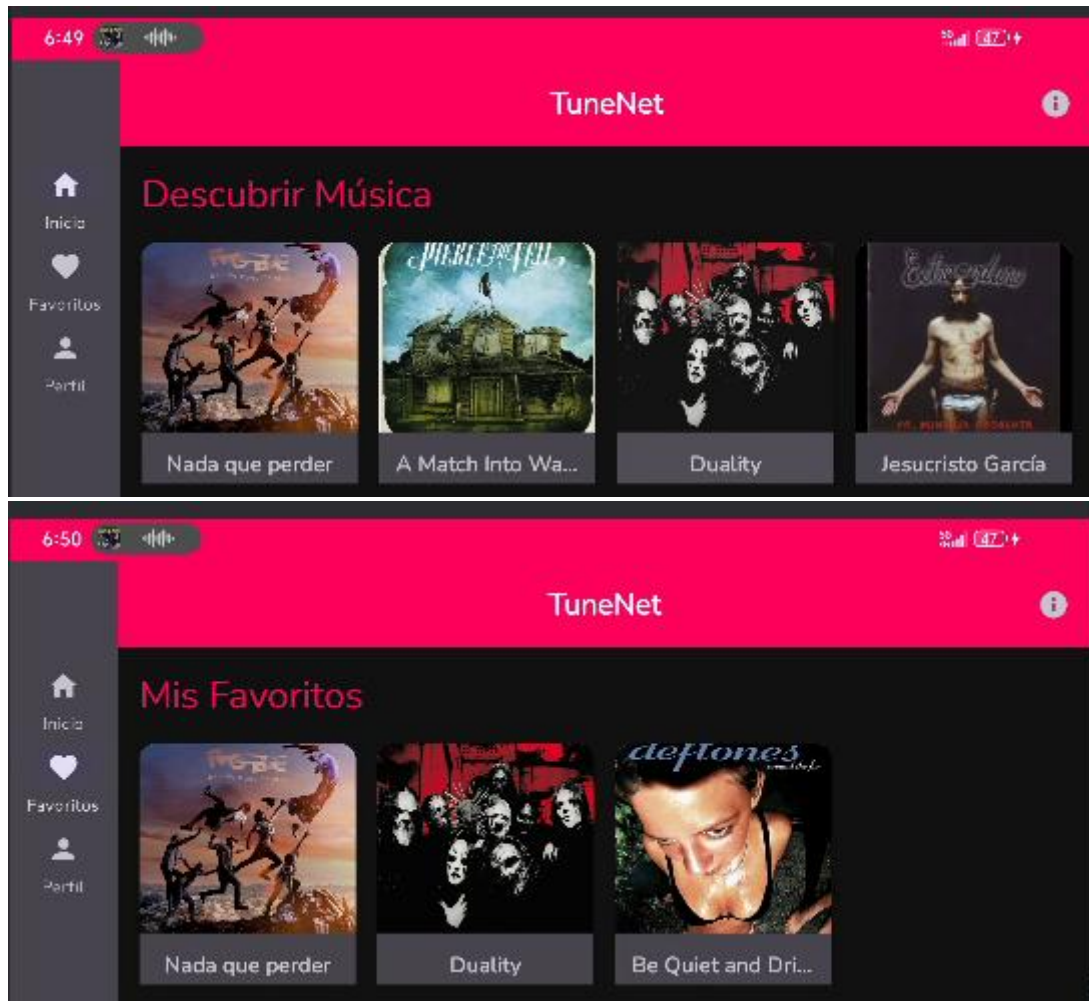
Con esta configuración consigo que la aplicación cumpla con los requisitos de **Responsive UI**, así ofreciendo una experiencia optimizada para cada tamaño de pantalla con una única base de código.

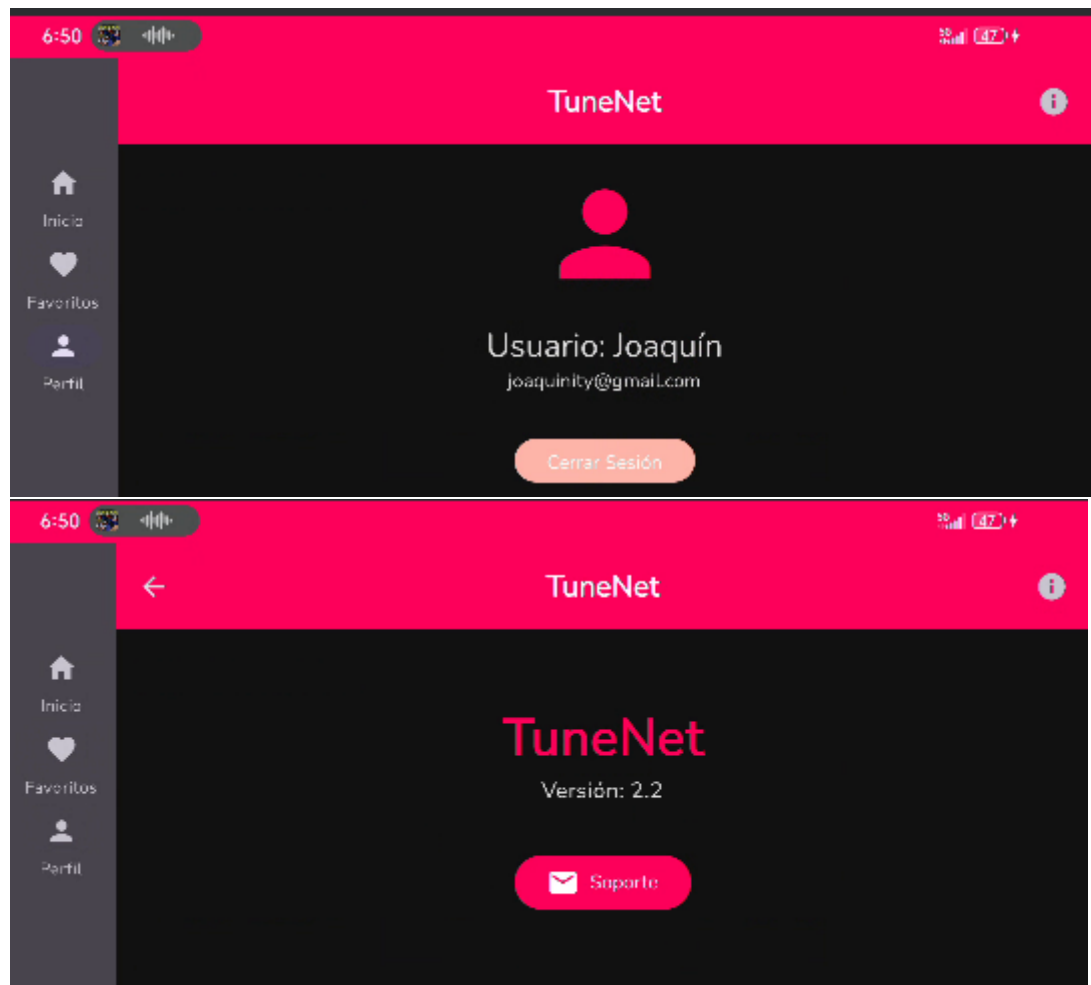
Comprobaciones

Móvil (Todas las pantallas)



Tablet





Y eso es todo lo que pide este ejercicio...